PrismPM closes a system inside the workshop. Hologram closes it outside. The third mirror takes the verified model and seals it as a portable object a foreign oracle can inspect, plan, and execute without trusting the builder. A JSON document, a mounted view, or a recording of a view is not that object.

## Profile is not container

Holo/1 is the Prism application profile defined by the generated declarations under `Foundation.Holo.V1`. It is not a version number. The physical container is a Hologram archive, version 4. Every `.holo` begins with `HOLO` followed by little-endian physical version 4 and canonical flags — the bytes `HOLO\x04\x00`. The stable Hologram build creates and validates v4 archives and rejects every other physical version. After the header come a section count and fixed 24-byte section-table entries: kind, offset, length.

The wire codec that writes those bytes is LexLean-generated `Foundation.Holo.V1.Wire` in `prism-stdlib`. It is not a vendored copy of the Hologram archive implementation. Neither Hologram platform code nor its archive implementation is a production Cargo dependency of PrismPM. Upstream compatibility oracles are pinned in `model/dependencies.toml`. The workshop may generate a guest the oracle understands. It may not link the oracle into itself and call that independence.

Two artifacts stay separate on purpose:

- `model.prism.json` is the canonical Prism model document. The projector is a total deterministic function of a valid LexLean snapshot ( `HO-04` ). Identity is SHA-256 ( `HO-03` ). Entity identifiers are qualified strings with deterministic zero-based indexes ( `HO-05` ). Legacy numeric applications and architecture models keep the closed `prismpm/model-document/1` schema and that filename ( `HO-02` ). The explicit text-application profile projects to `prismpm/model-document/2` and is not an additive change to `/1` ( `HO-11` ).
- `ApplicationName.holo` is the binary archive. JSON is never a `.holo`. Legacy JSON, including the prototype `prismpm/holo/1` document, is rejected, as are noncanonical archives, malformed values, and unsupported physical versions ( `HO-07` ).

The model document is the logical face. The archive is the physical face. Collapsing them would let a rewritten JSON impersonate an executable object.

## Three identities, not one hash

Hologram Live reports three identities, and PrismPM verification requires all three to match the current build:

| Identity | What it addresses | What may differ |
| --- | --- | --- |
| `archive_kappa` | BLAKE3 of the complete physical file | Fat and thin archives differ |
| `archive_fingerprint` | Footer integrity value | Fat and thin differ |
| `application_kappa` | Canonical `AppManifest` | Stable across fat and thin |

Fat is the default: content blobs embedded, executable without an external store. Thin emits the same manifest without its κ-addressed payloads and resolves them from a local kappa store after the fat archive has been imported. PrismPM's release object is the fat archive. A strict Calculator fat archive has one manifest, one Metadata section, two extensions — the upstream application directory and Prism provenance — four content blobs, and one footer. Unknown, duplicate, missing, reordered, overlapping, truncated, trailing, over-limit, or cross-reference-invalid content is rejected.

Kappa here is the content address: one address per object, BLAKE3 in the Live contract, the same addressing family UOR uses so identical bytes are not a second object. Archive kappa binds the file you were handed. Application kappa binds the application that file claims to be. Fingerprint binds the footer. Final archive fingerprint, archive kappa, and Prism attestation are bound after construction, so the provenance digest does not cycle.

The producer extension key is `https://uor.foundation/extension/prismpm-model/v1`. Its closed `prismpm/model-provenance/1` payload binds the authoritative model blob, LexLean and lean4-prod evidence, generated core, package, guest, View, and browser evidence, dependency commits, and application kappa. It cannot grant a capability or replace the manifest. Provenance is a witness. It is not an authority.

`HO-10` checks canonical sections, identities, content closure, directory derivation, and closed Prism provenance. `HO-06` excludes host paths, timestamps, and unstable environment data. `HO-09` binds generated Lean, the canonical model document, and the binary projection to the same sources through shared goldens. `HO-08` makes the emitter-semantics ID name the projector inputs, so a different projector cannot reuse an old document identity.

## The oracle is foreign on purpose

`prismpm verify` composes the v4 fat archive and then validates and executes it with the pinned Hologram Live oracle. Inspect and plan are the headless face:

```sh
```

```
hologram --json holo inspect path/to/Calculator.holo
hologram --json holo plan path/to/Calculator.holo
```

The ordinary headless plan reports that the portable View surface is unavailable. That report is not a failure of the archive. It is the plan telling the truth about the surface it was given. Verification builds the pinned independent oracle, opens the same archive with a display-independent portable surface, runs every modeled request directly and through View intents, then checks detach and idempotent shutdown.

Required portable-browser execution evidence uses `prismpm/hologram-oracle/2`. The prior recording-only `/1` report cannot satisfy current application verification. Its archived records remain historical evidence, not a browser execution claim. Calculator's oracle acceptance is 8 legacy numeric cases plus 10 UTF-8 text cases, recorded as `prismpm/holo-oracle-acceptance/1`. Execution evidence itself has schema `prismpm/execution-evidence/2`.

Browser evidence is stricter than a screenshot. Application verification must execute the exact portable View extracted from the archive, in pinned Chromium, with intents handled by the independent Hologram oracle session and its Core-Wasm runtime (`HO-12`). Recording that a View was mounted is not evidence that its JavaScript executed, or that the modeled request and response reached the rendered interface. Application kappa, archive kappa, and archive fingerprint on that evidence must equal the current build's Holo identities. Missing, extra, or inconsistent evidence fails with `PP5301`.

`HO-13` keeps the browser surface from becoming a side door. The modeled wire codec preserves legacy bytes, reproduces pinned upstream framing in generated std and `no_std` code, and rejects malformed or mixed profiles without granting browser authority. Caller cache overrides cannot select another binary. The shipped browser tree is read-only to unprivileged runtime users.

Hologram Live, as a product, is the local-first module host: CLI, background service, Desktop sidecar. It compiles `hologram.json` to v4, inspects without importing, plans viability, and runs Wasm through Wasmtime. Its layer set is closed — `wasm`, `tensor`, `rootfs`, `view`, `inference-model`. An `inference-model` layer may name engine `uor-r4` and can be imported without starting that engine. That host is the ecosystem the oracle is pinned from. It is not a second implementation of PrismPM, and PrismPM does not become a Hologram application by depending on it.

## What the Calculator archive actually holds

The production build's Hologram face is not the crate. It is `Calculator.holo`, the

The production build's hologram face is not the crate. It is `calculator.holo`, the composed Core-Wasm guest plus portable View, sitting beside `prism-calculator-0.1.0.crate`, `prism_calculator_core_wasm.wasm`, the generated View artifacts, and the model, Lean, LCNF, provenance, identity, and manifest evidence. The guest profile is import-free `hologram:guest/core-wasm@1`. The View has already been evaluated to portable HOLOVIEW before it is sealed. The archive is the composition, not a pointer back to the workshop tree.

Byte limits apply independently to the canonical model document and to each physical `.holo`, before atomic publication. The default project cap is 16 MiB, with a hard cap of 1 GiB. Over-limit content is rejected with the other structural faults.

## Why this is the third mirror

The first mirror made a sentence into a page and a proof. The second made a system into package, guest, view, and model document, under one build ID. The third makes that system into a file a party who did not run the workshop can address, plan, and execute.

Phase-mirror methodology at this joint is contractive in a specific way. The archive may not contain a path, a timestamp, a handwritten guest, or a provenance extension that grants a capability. Fat and thin may differ in archive kappa and fingerprint. They may not differ in application kappa. The oracle may refuse the archive. The workshop may not patch the oracle and keep the acceptance record. A Foundry site or workflow that has not been sealed this way can still be a model. It is not yet an object the civic base-state can hand to someone else.